INGENIERÍA DE SOFTWARE · UCAB · EQUIPO 87

Stack UcabMarket

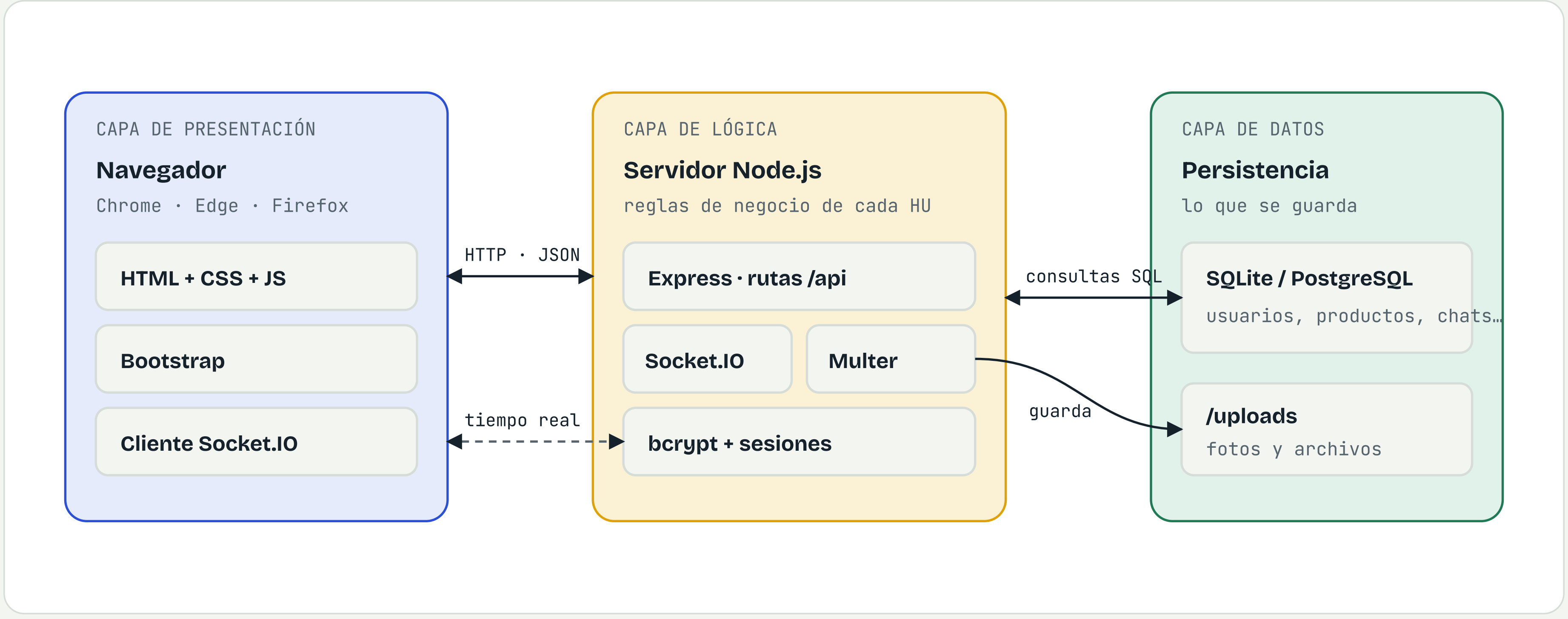
Un solo lenguaje de punta a punta: JavaScript en el navegador y en el servidor, con datos en JSON. Cinco personas aprendiendo una sola cosa, bien.

frontend
HTML · CSS · JS + Bootstrap
backend
Node.js + Express
datos
SQLite → PostgreSQL
`Socket.IO` `Multer` `bcrypt` `express-session`

CÓMO ENCAJA TODO

Arquitectura en tres capas

El navegador solo muestra y pide. El servidor decide (reglas de negocio, sesión, validaciones). La base de datos guarda. Cada capa se puede cambiar sin romper las otras, y eso es lo que luego permite aplicar tácticas arquitectónicas.



☐ lo que ve el usuario ☐ lo que decide el sistema ☐ lo que se guarda

LAS HERRAMIENTAS, UNA POR UNA

Qué usamos, por qué es buena y por qué la elegimos

FRONTEND

Lo que ve el miembro UCAB

HTML · CSS · JavaScript

estructura, estilo, lógica en pantalla

POR QUÉ ES BUENA

- Es el lenguaje nativo del navegador: no hay que compilar nada.
- Funciona igual en Chrome, Edge y Firefox.
- Todo lo demás de este stack se construye encima.

Por qué la elegimos: ya está en la tarjeta Herramientas del equipo y es lo primero que todos saben o pueden aprender rápido.

Descartamos React por ahora: es potente, pero suma una curva de aprendizaje que este proyecto no necesita. Si alguien del equipo ya lo domina, se puede reconsiderar.

Bootstrap

componentes de interfaz listos

POR QUÉ ES BUENA

- Trae botones, formularios, tarjetas, menús y modales ya diseñados.
- Rejilla para acomodar el catálogo en columnas sin pelear con CSS.
- Iconos propios (Bootstrap Icons) y documentación enorme.

Por qué la elegimos: una interfaz consistente y clara sin ser diseñadores. Apoya el rango de calidad **Capacidad de interacción**.

Descartamos Tailwind: obliga a armar cada componente con clases utilitarias; Bootstrap da el componente terminado.

BACKEND

Lo que decide el sistema

Node.js + Express

servidor y rutas de la API

POR QUÉ ES BUENA

- JavaScript también en el servidor: un solo lenguaje para todo el equipo.
- Express es mínimo: una ruta como `POST /api/chats` son pocas líneas.
- Recibe y responde JSON de forma natural.

Por qué la elegimos: aquí viven las reglas de negocio de cada HU (sesión iniciada, perfil dado de baja, máximo 5 fotos). Un backend propio deja ver la arquitectura y aplicar tácticas como el TimeOut del audio de clase.

Descartamos Firebase/Supabase: hacen el backend por ti, y justo eso es lo que la materia pide diseñar.

Socket.IO

chat en tiempo real

POR QUÉ ES BUENA

- Los mensajes llegan al instante, sin recargar la página.
- Se monta sobre el mismo servidor Express.
- Maneja reconexiones solo.

Por qué la elegimos: la meta Chats del Brief es «coordinar la compra por mensajes». Es la pieza de **Crear Chat** y **Añadir Archivos**.

Descartamos refrescar cada pocos segundos: más carga al servidor y un chat que se siente lento.

Multer

subida de fotos y archivos

POR QUÉ ES BUENA

- Recibe archivos en Express con una configuración corta.
- Limita cantidad, tamaño y tipo de archivo en un solo lugar.

Por qué la elegimos: cubre la restricción del Brief «máximo 5 fotos por publicación» (Añadir Producto) y los archivos del chat (Añadir Archivos).

bcrypt + express-session

seguridad de las cuentas

POR QUÉ ES BUENA

- bcrypt guarda las claves cifradas: ni el equipo puede leerlas.
- express-session recuerda quién inició sesión.

Por qué la elegimos: el rango de **Seguridad → Confidencialidad**: «que la información confidencial de las cuentas esté resguardada». También permite aceptar solo correos UCAB al crear cuenta.

DATOS

Lo que se guarda

SQLite → PostgreSQL

base de datos relacional

POR QUÉ ES BUENA

- SQLite es un solo archivo: cero instalación para empezar.
- PostgreSQL es la versión para publicarlo en internet; el SQL casi no cambia.
- Tablas y relaciones: usuarios, productos, preguntas, chats, mensajes, reseñas.

Por qué la elegimos: los datos de UcabMarket están muy relacionados (un chat es de un producto y de dos miembros). Una base relacional lo modela directo, y sirve para el ítem «Estudio de datos» del Flujo de Trabajo.

Descartamos guardar todo en archivos JSON: sirve para un prototipo, pero se corrompe cuando dos personas escriben a la vez. JSON queda para intercambiar datos entre capas.

TRABAJO Y DISEÑO

Con qué trabaja el equipo

VS Code + Live Server

editor

POR QUÉ ES BUENA

- Gratis, rápido, con extensiones para todo el stack.
- Live Server recarga la página al guardar.
- Thunder Client (extensión) prueba la API sin salir del editor.

Git + GitHub

repositorio del equipo

POR QUÉ ES BUENA

- Cinco personas en el mismo código sin pisarse.
- Historial de cada cambio, como el histórico de versiones del Brief.
- GitHub Desktop si la consola intimida.

Por qué la elegimos: es el «Repositorio Git» que ya está en la tarjeta Herramientas.

Figma

diseño de la interfaz

POR QUÉ ES BUENA

- Pantallas clicables antes de programar.
- Los cinco editan el mismo archivo a la vez.
- Trae kits de componentes de Bootstrap, así lo diseñado coincide con lo programado.

Por qué la elegimos: es el ítem «Diseño de la interfaz (HM)» del Flujo de Trabajo, y la profesora pidió que el producto coincida con el plano: «si hiciste un rancho, tiene que ser un rancho».

draw.io · Canva

diagramas y backlog

POR QUÉ ES BUENA

- draw.io: diagramas de actividades, casos de uso y arquitectura, con enlace para compartir.
- Canva: el backlog (story map) que ya tienen.

Netlify · Render

publicarlo en internet

POR QUÉ ES BUENA

- Planes gratuitos para proyectos de estudiantes.
- Netlify sirve el frontend; Render corre el servidor Node.js.
- Se actualizan solos al subir cambios a GitHub.

SPRINT 1

Qué herramienta pide cada historia

HISTORIA DE USUARIO	BOOTSTRAP	EXPRESS	BASE DE DATOS	SOCKET.IO	MULTER	BCRYPT / SESIÓN
Crear Cuenta Paola	●	●	●			●
Consultar Perfil Paola	●	●	●			○
Añadir Producto Victoria	●	●	●		●	○
Consultar Producto Victoria	●	●	●			
Añadir Preguntas Eliavalentina	●	●	●			○
Consultar Preguntas Eliavalentina	●	●	●			
Crear Chat José	●	●	●	●		●
Añadir Archivos José	●	●	●	●	●	○
Añadir Reseña Gabriel	●	●	●			○
Consultar Reseña Gabriel	●	●	●			

● la usa directamente ○ solo verifica la sesión iniciada

ISO/IEC 25010

Cómo el stack cumple los rangos de calidad del Brief

Capacidad de interacción → Operabilidad

Sencilla de usar

«Que la aplicación sea sencilla de usar»

Componentes de Bootstrap reconocibles, diseño probado en Figma antes de programar y mensajes claros en español.

Eficiencia de desempeño → Comportamiento temporal

Catálogo rápido

«Rápida en las funcionalidades del catálogo de productos»

Paginación del catálogo e índices en la base de datos; Express responde solo los datos de la página que se ve.

Seguridad → Confidencialidad

Cuentas resguardadas

«Que la información confidencial de las cuentas esté resguardada»

Claves cifradas con bcrypt, sesiones en el servidor y el chat sin mostrar correo ni teléfono.

Flexibilidad → Adaptabilidad

Varios navegadores

«Que pueda abrir en varios navegadores web»

HTML, CSS y JavaScript estándar con Bootstrap: funcionan igual en Chrome, Edge y Firefox de escritorio.

PARA ARRANCAR

Primeros pasos y estructura del proyecto

Instalar y crear el proyecto

1. Instalar **Node.js** (versión LTS), **VS Code** y **Git**.
2. Crear el repositorio en GitHub y clonarlo.
3. Dentro de la carpeta del proyecto:

```
npm init -y
npm install express socket.io multer bcrypt express-s
```

4. Cada integrante trabaja su HU en una rama propia y la une con un pull request.

Carpetas por capa

```
ucabmarket/
├ public/           ← presentación
│  ├ index.html     catálogo
│  ├ producto.html  detalle + «Contactar vendedor»
│  ├ chat.html
│  ├ css/  js/
├ src/              ← lógica
│  ├ server.js      Express + Socket.IO
│  ├ routes/        productos, chats, reseñas…
│  └ middleware/    sesión, Multer
├ db/               ← datos
│  └ ucabmarket.sqlite
└ uploads/          fotos y archivos
```

> **Propuesta para decidir en equipo.** Antes de cambiar la tarjeta Herramientas del Trello, confírmenla entre los cinco. Si la profesora pide una tecnología concreta, esa indicación manda sobre este documento.

UcabMarket · Equipo 87 · Ingeniería de Software, UCAB · Base: Brief v1.6 (16/10/2026) y backlog del Sprint 1.